Fixed programs have few shapes: a reduction of the Main Conjecture to a counting problem

Research note accompanying the uniform-process construction

October 20, 2026

Abstract

A computation from the remainder function determines a *shape*: the straight-line sequence of remainder instructions it performs, each operand named by where it came from. We show that a fixed recursive program, deterministic or nondeterministic, produces at most C_P^{c+1} shapes among its computations with at most c remainder calls. Unrestricted derivations have about c^{2c} shapes. Under the hypothesis below, this difference accounts for the whole gap between $\log a$ and $\log a/\log\log a$. Both open questions left by the companion manuscript [2] would be settled by a single counting hypothesis: how many pairs $(a,b) \le N$ can one shape of length d bring to the value 1? If no shape of length at most $c_0 \log N$ succeeds on more than $N^{2-\delta}$ pairs, then every fixed program makes at least $r_P \log a$ remainder calls on almost all coprime pairs. That is the Main Conjecture, in a strong form. The same hypothesis also makes the companion bound $O(\log a/\log\log a)$ tight for almost all coprime pairs. Without any hypothesis, a single program with $o(\log a)$ calls would force shapes of length $o(\log N)$ that succeed on $N^{2-o(1)}$ pairs. The Shape-Count Lemma and both reductions are formalized in Lean; the counting hypothesis is open.

1 Shapes

In this note log is the natural logarithm, as in the Lean statements. The companion manuscript uses base 2; this rescales only the constants c_0 and r_P below, and $\log N/\log\log N$ is asymptotically independent of the base.

A *shape* of length d is a sequence $S = ((i_1, j_1), \dots, (i_d, j_d))$ with $i_t, j_t \in \{0, 1, \dots, t\}$. On an input (a, b) put $v_0 = a$, $v_1 = b$ and $v_{t+1} = \mathrm{rem}(v_{i_t}, v_{j_t})$. The shape is *valid* on (a, b) if every divisor v_{j_t} is nonzero. It *reaches* 1 if moreover some $v_{t+1} = 1$. Thus a remainder-only derivation in the sense of [2] is exactly a valid shape. Let

$$P_S(N) = \#\{(a,b) : 2 \le b < a \le N,\ S \text{ is valid on } (a,b) \text{ and reaches } 1\}.$$

There are $\prod_{t=1}^{d}(t+1)^2 = ((d+1)!)^2$ shapes of length d. This is the count that underlies $\log a/\log\log a$.

2 The shape count of a fixed program

2.1 Programs and computations

We make the operational model explicit. It is the standard call-by-value reading of the recursive programs of [1, §2A, §2E], and we take it as the definition of a computation and of its cost.

Structure. The structure is $(\mathbb{N}, \mathrm{rem}, \mathrm{eq}_0)$ or $(\mathbb{N}, \mathrm{rem}, \mathrm{eq}_0, \mathrm{eq}_1)$. There are two sorts, ι (natural numbers) and $\mathbf{b}$ (truth values tt, ff). $\mathrm{rem}(x, y)$ is defined only for $y > 0$, and $\mathrm{eq}_0(x)$, $\mathrm{eq}_1(x)$ test $x = 0$ and $x = 1$. The structure has no constants of sort ι, so a program has none either. This matters: a constant 1 would make every shape question trivial.

Programs. A program P has finitely many function symbols p, each with an arity k_p, argument sorts and a result sort. It has finitely many equations $p(x_1, \ldots, x_{k_p}) = e$, possibly several for the same p (nondeterminism), and a main term e_0 in the input variables x_1, x_2. Terms are built from variables, the constants tt, ff, the primitives $\mathrm{rem}(s, t)$, $\mathrm{eq}_0(s)$, $\mathrm{eq}_1(s)$, conditionals if s then t else u with s of sort $\mathbf{b}$, and calls $p(s_1, \ldots, s_{k_p})$, all well sorted. The variables of a body are among its equation's parameters. Let $\mathcal{E}$ be the finite set of subterms of bodies and of e_0. Let $K \geq 2$ bound the arities, and let A bound the number of immediate subterms of a term in $\mathcal{E}$. A call has its arguments as immediate subterms, plus one more child, the body.

Computations. An *environment* assigns values to the variables of an equation (or of e_0). A *computation tree* is a finite tree whose nodes are triples (environment ρ, term $e \in \mathcal{E}$, value v), built by these rules.

- **Variables and constants:** a leaf with $v = \rho(x)$, or v the constant.
- **Primitives:** for $\mathrm{rem}(s, t)$, $\mathrm{eq}_0(s)$, $\mathrm{eq}_1(s)$ the children are computations of s (and t) in ρ with values x (and y). For rem we need $y > 0$, and then $v = \mathrm{rem}(x, y)$. For the tests, v is the truth value of $x = 0$ or $x = 1$.
- **Conditionals:** the children are a computation of s in ρ with value β, and a computation of t (if $\beta = \mathrm{tt}$) or of u (if $\beta = \mathrm{ff}$) in ρ whose value is v.
- **Calls:** for $p(s_1, \ldots, s_k)$ the children are computations of each s_i in ρ with values w_i, then a computation of the body e of *some* equation for p, in the environment $x_i \mapsto w_i$, whose value is v.

A *convergent computation* of P on (a, b) is a computation tree for e_0 in $x_1 \mapsto a$, $x_2 \mapsto b$. Its value is the value at the root, and its *cost* is its number of rem nodes.

- **Correctness:** following the book's convention for nondeterministic programs, P *computes* f if every convergent computation has the correct value and one exists on every input.
- **Complexity:** $c_P(a, b)$ is the least cost of a convergent computation on (a, b).

Divergence never enters: a convergent computation is a finite tree, so any amount of free, cost-zero recursion in it is finite. Executing the tree in post-order is what the recursive machine does.

2.2 Provenance and shapes

Fix a convergent computation $\mathcal{T}$ on (a, b). List its rem nodes in post-order as $n_1, \ldots, n_c$. Every ι-valued node z gets a *provenance* $\pi(z) \in \{0, \ldots, c+1\}$, and every environment entry of sort ι gets one too, by induction along the tree:

- the root environment gives x_1 provenance 0 and x_2 provenance 1;
- a variable leaf has the provenance of its environment entry;

- the node n_t has provenance $t+1$;
- a conditional has the provenance of its chosen branch;
- a call has the provenance of its body child;
- the body's environment entry x_i has the provenance of the i-th argument child.

The *shape* of $\mathcal{T}$ is $S(\mathcal{T}) = ((\pi(\ell_1), \pi(r_1)), \ldots, (\pi(\ell_c), \pi(r_c)))$, where ℓ_t, r_t are the two children of n_t. By induction along the tree, every provenance carried by a node or an environment entry is 0, 1, or $s+1$ for a rem node n_s that is completed before that node or entry is used: a body's environment entries come from argument children, which precede the body in post-order. The operands ℓ_t, r_t are completed before n_t, so $\pi(\ell_t), \pi(r_t) \leq t$, and this is a shape.

Lemma 1 (Soundness)**.** *Run $S(\mathcal{T})$ on (a, b) with values $v_0, \ldots, v_{c+1}$. Every ι-valued node z has value $v_{\pi(z)}$, and the same holds for every ι-valued environment entry. In particular $S(\mathcal{T})$ is valid on (a, b), and every ι-value occurring in $\mathcal{T}$ is one of the v_i.*

Proof. Induction in post-order. Each rule for π copies a value together with its provenance. At n_t the children have values $v_{\pi(\ell_t)}$ and $v_{\pi(r_t)}$, and the latter is positive because the rem rule requires it. So the node's value is $\mathrm{rem}(v_{\pi(\ell_t)}, v_{\pi(r_t)}) = v_{t+1}$. □

Lemma 2 (Light subtrees select)**.** *Call a node* light *if its subtree contains no* rem *node. A light ι-valued node z evaluated in environment ρ has $\pi(z) = \pi(\rho(x))$ for some variable x of ρ.*

Proof. Induction on the subtree. A variable leaf is immediate. A light node cannot be a rem node, and no constant has sort ι. A conditional inherits from its chosen branch, which is evaluated in ρ. A call inherits from its body, which is evaluated in an environment whose entries are the arguments. The arguments are light ι-valued nodes in ρ, so by induction each has the provenance of a variable of ρ. □

2.3 The Shape-Count Lemma

Lemma 3 (Shape-Count Lemma)**.** *There is a constant Σ_P, depending only on P, such that the shapes of convergent computations of P with cost c, on all inputs together, number at most $(2\Sigma_P)^{4c+2}$. Hence those of cost at most c number at most C_P^{c+1}, with $C_P = (2\Sigma_P)^5$.*

Proof. Let $\mathcal{T}$ have cost c. Call a node *heavy* if it is not light. Every ancestor of a heavy node is heavy, so for $c \geq 1$ the heavy nodes form a subtree H containing the root. Its leaves are rem nodes. Fewer than c of its nodes have two or more heavy children ("branching" nodes), because a tree has fewer branching nodes than leaves. Call every other non-rem node of H a *chain node*: it has exactly one heavy child.

The data at a node. For a node z of H evaluated in environment ρ_z with variables $x_1, \ldots, x_k$ ($k \leq K$), let the *label* of z record:

- the term $e_z \in \mathcal{E}$;
- for a call, which equation was used;
- for each child of z (at most $A+1$ of them), whether it is heavy, and if it is light and ι-valued, an index $i \leq k$ with $\pi(\text{child}) = \pi(\rho_z(x_i))$ (Lemma 2). For the body child of a call, which lives in the new environment, record instead which argument position supplies its provenance.

Given the provenances of ρ_z and of the results of the heavy children of z, the label determines the provenances of the following, by the rules defining π:

- the environments of all children of z;
- the result of z;
- for a rem node, its operand pair.

There are at most $\Lambda_P = |\mathcal{E}| \cdot (\#\text{equations}) \cdot (2(K+1))^{A+1}$ labels.

Contracting chains. Take a maximal path $z_0 \to z_1 \to \cdots \to z_m$ in H whose interior nodes $z_1, \ldots, z_{m-1}$ are chain nodes. All siblings along it are light, so by induction along the path two things hold:

- every ι-entry of ρ_{z_m} has the provenance of an entry of ρ_{z_1}, which gives a map $\phi\colon \{1..K\} \to \{1..K\}$;
- the result of z_1 has the provenance either of the result of z_m or of an entry $\rho_{z_1}(x_i)$, or it is a truth value. That is a datum $\psi \in \{1..K\} \cup \{\text{bottom}, \text{bool}\}$.

Contract the path to an edge labelled (ϕ, ψ). There are at most $K^K(K+2)$ edge labels. The contracted tree H' has the following nodes: the c rem nodes, fewer than c branching nodes, and the root of H if it is a chain node. So it has at most $2c$ nodes and at most $2c$ edges.

Encoding. Write H' in preorder, each node written as its label and followed by the labels of the edges to its children, with the children in their order in $\mathcal{T}$. This gives a word of length at most $2c + 2c \le 4c + 2$ over an alphabet of size $\Sigma_P = \Lambda_P + K^K(K+2)$.

The word determines $S(\mathcal{T})$. Decode recursively in evaluation order, starting at the root with provenances $0, 1$ for x_1, x_2: given the provenances of ρ_z, treat the children of z in order. The label of z gives the environment provenances of each child from those of ρ_z and, for a body, from the results of the argument children already decoded. An edge label transports environment provenances down a contracted chain and the result provenance back up. Decoding a heavy child returns its result provenance, and the label then gives that of z. So every rem node's operand pair is determined. The post-order of the rem nodes is also determined, since light subtrees contain none. For $c = 0$ the shape is empty. Finally, there are at most $\sum_{\ell \le 4c+2} \Sigma_P^\ell \le (2\Sigma_P)^{4c+2}$ words of length at most $4c + 2$. □

The proof uses only finiteness of $\mathcal{E}$, bounded arity, and the absence of ι-constants; the Lean version below shows that bounded arity alone suffices, with a weaker constant. Unbounded cost-zero recursion, truth values, conditionals and nondeterministic choice are all absorbed into the labels.

Remark 4 (Register programs). The constant is crude, but its order can be pinned down for register programs. Consider runs of width W whose steps overwrite a register by a remainder of two registers, route registers (any map of registers to registers), or run a subcomputation on a copy of the registers and overwrite one register by its result; the program of [4] is of this kind. We sketch why such runs have at most $(8W^3)^{5c+1}$ shapes with c remainder calls; this sketch is not formalized.

To see this, name each live value of a frame, rather than each register, by a label in $\{1, \ldots, W\}$; there are at most W live values. Encode a remainder step by the labels of its two operands and the label given to the result, which takes over the label of a value never used again in this frame (or a free label). Routes need no encoding, since labels follow values. Dropping a value needs no encoding either, since only operands are decoded. A subcomputation inherits the labels and is encoded by

an opening marker, its own steps, and a closing marker carrying the label it returns and the label the result takes in the caller (or a mark that the result is discarded). Subcomputations without remainder calls only copy a value, and are absorbed as routes. A subcomputation consisting only of routes and one inner subcomputation is replaced by that inner one: its returned value is either the inner result, whose label it takes, or a copied caller value, in which case the inner result is discarded. After these reductions, in the tree of subcomputations every leaf performs a remainder step of its own, and every node without one has at least two children. So there are at most c of the former and fewer than c of the latter, at most $2c$ subcomputations, and at most $5c$ steps and markers in all. With an end marker, each symbol has one of four types and at most W^3 choices of labels, and words of length at most $5c+1$ number at most $(8W^3)^{5c+1}$. The labels determine every operand's provenance. Hence $\log C_P = O(\log W)$ for such programs.

3 Reductions

Theorem 5 (Reduction)**.** *Let P compute $\gcd(a,b)$ over $(\mathbb{N}, \mathrm{rem}, \mathrm{eq}_0)$, or decide coprimeness over $(\mathbb{N}, \mathrm{rem}, \mathrm{eq}_0, \mathrm{eq}_1)$, for $a > b \geq 2$, and let $c_P(a,b)$ be its remainder complexity. Then for all N and c,*

$$\#\{(a,b) : 2 \leq b < a \leq N,\ \gcd(a,b) = 1,\ c_P(a,b) \leq c\} \ \leq\ C_P^{c+1} \cdot \max_{|S| \leq c} P_S(N).$$

Proof. Take a convergent computation with $c_P(a,b)$ remainder calls, and let S be its shape. S is valid on (a,b), and it reaches 1. For the gcd, the output 1 is a value of the computation and not an input, since $a > b \geq 2$. For coprimeness, suppose no value of the computation equals 1. Doubling every value preserves every remainder equation ($\mathrm{rem}(2x, 2y) = 2\,\mathrm{rem}(x,y)$), every eq_0 answer, and every eq_1 answer (all false), and conditionals take the same branches. So the doubled tree is a convergent computation at $(2a, 2b)$ answering "coprime". That is wrong, since $\gcd(2a, 2b) \geq 2$, and P is correct on every convergent computation. Hence some value is 1. By Lemma 1 it is some v_i, and since $a, b \geq 2$ it is a produced value v_{t+1}.

Sum over the at most C_P^{c+1} shapes of Lemma 3. □

Since about $3N^2/\pi^2$ pairs in the range are coprime, we get the following.

Corollary 6 (A single sublogarithmic program needs very lucky shapes)**.** *If one program P has $c_P(a,b) \leq f(a)$ for all coprime $a > b \geq 2$, with f nondecreasing and $f(N) = o(\log N)$, then for every $\varepsilon > 0$ and all large N some shape of length at most $f(N)$ reaches 1 on at least $N^{2-\varepsilon}$ pairs $(a,b) \leq N$.*

Conjecture 7 (Per-shape bound, PSB(c_0, δ))**.** *There are $c_0, \delta > 0$ such that $P_S(N) \leq N^{2-\delta}$ for every large N and every shape S of length at most $c_0 \log N$.*

Some restriction on the length is necessary. For fixed d, Euclid's shape of length d first reaches 1 on about $N(\ln N)^d/d!$ pairs. At lengths proportional to $\ln N$ it does far better. By Dixon's theorem [3], for all but $o(N^2)$ pairs with $N/2 < a \leq N$ the number of Euclidean steps is $(12 \ln 2/\pi^2) \ln N + O((\ln N)^{1/2+\varepsilon})$. By pigeonhole, there is a single $d = (12 \ln 2/\pi^2 + o(1)) \ln N$ such that on $N^{2-o(1)}$ coprime pairs the Euclidean remainders first reach 1 after exactly d steps, and Euclid's shape of length d reaches 1 on all of them. So PSB(c_0, δ) fails for every $c_0 > 12 \ln 2/\pi^2$.

Corollary 8 (PSB implies the Main Conjecture for programs)**.** *Assume* PSB(c_0, δ)*. For every program P as in Theorem 5, put $r_P = \min\{c_0,\ \delta/(2 \log C_P)\}$. Then $c_P(a,b) \geq r_P \log a$ for all but $o(N^2)$ coprime pairs $2 \leq b < a \leq N$. In particular, P makes at least $r_P \log a$ remainder calls on infinitely many pairs, deterministic or not.*

Proof. Take $c = \lfloor r_P \log N \rfloor \le c_0 \log N$. Then $C_P^{c+1} \le C_P\, N^{r_P \log C_P} \le C_P\, N^{\delta/2}$, so Theorem 5 and PSB give at most $C_P \cdot N^{\delta/2} \cdot N^{2-\delta} = O(N^{2-\delta/2})$ exceptions below N. Finally $\log a \le \log N$. □

Corollary 9 (PSB implies tightness)**.** *Assume* PSB(c_0, δ) *and let* $0 < \kappa < \delta/2$*. Then the companion's* $D(a,b)$ *satisfies* $D(a,b) > \kappa \log N / \log\log N$ *for all but* $o(N^2)$ *coprime pairs* $2 \le b < a \le N$*. Since all but* $o(N^2)$ *of these pairs have* $a > N/\log N$*, so that* $\log a \sim \log N$*, together with the companion's upper bound this gives* $D = \Theta(\log a / \log\log a)$ *on almost all coprime pairs.*

Proof. Put $d = \lfloor \kappa \log N / \log\log N \rfloor$; for large N, $d \le c_0 \log N$. Shapes of length at most d number at most $(d+1)((d+1)!)^2 \le (d+2)^{2d+1} = N^{2\kappa + o(1)}$. A coprime pair with $D(a,b) \le d$ is one on which some such shape reaches 1, so by PSB there are at most $N^{2-\delta+2\kappa+o(1)} = o(N^2)$ of them. □

4 Consistency with what is known

- **Upper bounds.** The companion construction uses about c^{2c} shapes, and every fixed-width program of [4] has a coefficient $3/p$ with width $W = 4^p + 5$, that is $\Theta(1/\log W)$. By Remark 4, Corollary 8 would give a matching $\Omega(1/\log W)$ for register programs of width W. So, granted PSB, the family of [4] is optimal up to a constant factor, and the quantifier exchange $\forall\varepsilon\,\exists P \to \exists P\,\forall\varepsilon$ fails.
- **Pratt's algorithm.** It uses only $O(\log\log a)$ remainder calls on successive Fibonacci numbers [1, Corollary 6C.7]. That set is sparse, so this is consistent with the almost-all bound of Corollary 8.
- **The homomorphism method.** Corollary 8 is not a bound for all uniform processes, since uniform processes are not limited to C^c shapes. So it does not conflict with the companion's limit on the homomorphism method.
- **The stack.** Recursion can keep older values alive, which suggested that one program might emulate the growing store of the companion construction. Lemma 3 shows this cannot multiply the number of shapes beyond C_P^{c+1}: a call returns one value, and everything else in its frame is lost.

5 Status

Lemma 3, Theorem 5 and Corollary 6 are unconditional. Corollaries 8 and 9 depend on Conjecture 7, which is open. The lemma is proved for the explicit operational model of Section 2.1: two sorts, no ι-constants, call-by-value calls, lazy conditionals, nondeterministic choice of equations, convergent computations as finite trees, and cost equal to the number of rem nodes. That general form is also formalized in Lean (`ShortGCD/GeneralPrograms.lean`), and all modules named below are part of the repository's normal `lake build` and axiom audit.

- `compile`: every evaluation of a program with arities at most $K \ge 2$ compiles into a register run of width $2K+2$ with the same number of remainder calls. Each subterm is evaluated in a subcomputation on a copy of the registers, so the register-program argument below applies.
- `eval_shape` and `card_shapeSet_le_pow`: every evaluation on inputs $a, b \ge 2$ returning 1 with at most c calls has a shape of length at most c reaching 1, drawn from a fixed set of at most $(7A_{2K+2}^5)^{c+1}$ shapes, with A_W as below. This is the Shape-Count Lemma with $C_P = 7A_{2K+2}^5$, which depends only on K.

- `eval_shape_val`: the provenance of the output.
- `card_eval_le` and `eventually_card_eval_le`: the counting consequences.
- `double_or_one`, `decide_run`, `card_decide_le` and `eventually_card_decide_le`: the doubling argument and the finite and asymptotic counting for programs that decide coprimeness. Correctness is assumed only on inputs $a > b \geq 2$, as in Theorem 5; there the book's relation agrees with $\gcd(a, b) = 1$.

The formal model is the one of Section 2.1, except that values are tagged, and both the set of function symbols and the set of bodies may be arbitrary rather than finite. So ill-sorted terms simply have no evaluation, and the constant depends only on the maximal arity. The formal constant is of order $(2K + 2)^{10K+10}$ rather than the finite-alphabet constant of Lemma 3. That is harmless for the Main Conjecture, but too weak for the optimality remark about register programs.

The register-program case is formalized in Lean, in `ShortGCD/ShapeCount.lean`, for the width-W run semantics `RegisterProgram.Run` of [4].

- `card_run_le` is the finite counting statement. Suppose every shape of length at most c reaches 1 on at most B pairs $2 \leq b < a \leq N$. Then at most $(5c + 2)\, A_W^{5c+1} B$ such pairs admit a run returning 1 with at most c remainder calls. Here $A_W = W^W + W^3 + W^2 + W + 1$ is the size of a fixed instruction alphabet.
- `card_accepts_le` gives the same bound for the full verified program.
- `eventually_card_run_le` is the asymptotic form. $\mathrm{PSB}(c_0, \delta)$, stated with natural logarithms, gives for each width an $r > 0$ with at most $N^{2-\delta/2}$ such pairs for all large N when the budget is $r \ln N$ calls.

The proof compiles a run into a linear code of at most $5c + 1$ instructions. Routes are merged and zero-cost subcomputations are absorbed. The shape is then read off by symbolic execution of the code. The axiom audit lists only `propext`, `Classical.choice` and `Quot.sound`.

The formalized constant gives $\log C = O(W \log W)$, since the alphabet includes all W^W routes. The sharper $O(\log W)$ of Remark 4 is not formalized.

Corollary 9 is formalized in `ShortGCD/TightnessReduction.lean`. `card_within_le_shapes` bounds the number of pairs with a derivation of 1 of length at most d by $(d + 1)(d + 2)^{2d} \max_{|S| \leq d} P_S(N)$, and `eventually_card_within_le` states the conclusion with natural logarithms: if every shape of length at most $\kappa \ln N / \ln \ln N$ reaches 1 on at most $N^{2-2\kappa-\delta}$ pairs, then eventually at most $N^{2-\delta/2}$ pairs have a derivation of 1 of length at most $\kappa \ln N / \ln \ln N$. `AlmostAll.within_iff_D_le` identifies derivations of 1 with D on coprime pairs.

References

[1] Yiannis N. Moschovakis. *Abstract Recursion and Intrinsic Complexity*. Lecture Notes in Logic 48, Cambridge University Press / ASL, 2019. doi:10.1017/9781108234238.

[2] Jason Davies. *The Euclidean algorithm is not optimal among uniform processes*. Companion repository manuscript, October 2026.

[3] John D. Dixon. The number of steps in the Euclidean algorithm. *Journal of Number Theory* 2(4):414–422, 1970.

[4] Jason Davies. *Fixed-arity nondeterministic remainder programs: an intermediate result.* Research note in the companion repository, `research/nondeterministic-prefix.tex`.